Loc-Camel-Route

Alberto Espinosa (KS-Group)

Oct 19th, 2025

Abstract

This document describes a contract-first, BPEL-driven integration sample and its transformation into a Spring + Maven + Camel implementation using local agentic tooling. It clarifies the problem, the original technology stack and artefacts, the proposed agent architecture and information flow, and the expected results. It also defines project organisation and installation scripts required to run the pipeline without mocks, using only real endpoints and data.

1 Problem Statement

The sample contains SOAP (Simple Object Access Protocol: XML-based messaging protocol for web services) WSDL (Web Services Description Language: XML format describing service interfaces)/XSD (XML Schema Definition: XML format defining data structures) contracts and a BPEL (Business Process Execution Language: XML language for orchestrating web-service workflows) process *(LocationRetrievalLOC3X1Process)* that orchestrates two inbound operations and multiple partner service calls. There is no runnable Java build, no proper server host, and legacy Java snippets are considered non-authoritative. The problem is to translate this into a robust Spring Boot (Java micro-service framework) + Maven (Java build/dependency tool) project with Apache Camel (integration routing engine)/CXF (Java web-service framework), regenerate models and stubs from contracts, implement deterministic mappings that replace BPEL assignments, externalise configuration, and provide contract and integration verification. The transformation must be automated by local agents and verified end-to-end without mocks; integration results depend on live endpoints.

2 Original Technology and Artefacts

2.1 Overview

The original sample is contract-first and IBM BPEL-centric:

- WSDL/XSD defining document/literal SOAP operations for Location and Country services.
- BPEL process (IBM WebSphere style) with partner links, variables, fault handlers, and embedded Java logging/fault composition.
- HTTP binding WSDLs referencing localhost endpoints.
- Shared fault schema (SimpleFault.xsd).

2.2 Organisation Diagram

Figure 1 illustrates the folder organisation of the original sample.

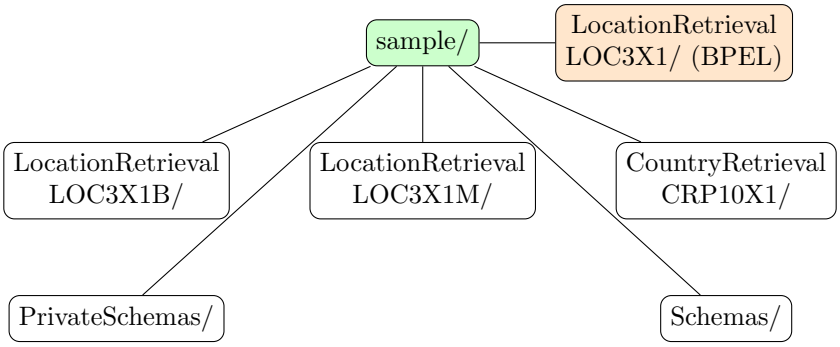

Figure 1: Folder organisation of the original sample.

3 File Structure and Standards

3.1 Summary Table

Table 1 summarises the key folders and files, their types, and roles.

Table 1: File structure summary of the original sample.

Folder/File	**Type**	**Role**
LocationRetrieval LOC3X1 / LocationRetrieval LOC3X1Process.bpel	BPEL	Orchestration: pick/onMessage for inbound operations, partner links (LOC3X1M, CRP11X1, CRP10X1), variables, fault handlers, logging.
LocationRetrieval LOC3X1B / LocationRetrieval-LOC3X1B.wsdl	WSDL	Public interface: LOC3X1B operations, request/response messages, faults.
LocationRetrieval LOC3X1M / LocationRetrieval-LOC3X1M.wsdl	WSDL	Internal interface: LOC3X1M operations to which requests are transformed.
CountryRetrieval CRP10X1 / CountryRetrieval-CRP10X1.wsdl	WSDL	Partner interface: country retrieval operation(s).
PrivateSchemas / SimpleFault.xsd	XSD	Shared fault structure (FaultMessageText).
Schemas/*	XSD	Domain types referenced by requests/replies across services.

File types: functions, protocols, and standards

- **BPEL (Business Process Execution Language)**: Defines executable orchestration of service interactions (picks, onMessage, invoke, assign, fault handlers). It runs on a BPEL engine and coordinates partner links and variables to implement business processes, typically calling SOAP-based services described by WSDL. Common deployment uses HTTP(S) transport and adheres to vendor-specific extensions (e.g., IBM WebSphere styles) while following WS-BPEL principles.

- **WSDL (Web Services Description Language)**: Specifies service contracts, including operations (portTypes), request/response messages, bindings, and service endpoints. In this sample WSDLs are document/literal SOAP with HTTP(S) bindings and include fault definitions. Good practice is alignment with WS-I Basic Profile and consistent namespaces and versions for cross-service compatibility.
- **XSD (XML Schema Definition)**: Provides formal XML type definitions for messages and domain objects used by WSDL services and BPEL variables. XSDs enforce structure and validation, carry namespaces for modularity, and support versioning. The shared SimpleFault.xsd standardises fault payloads across services, enabling consistent error reporting.

4 Project Description

4.1 Objectives

Create an automated, agent-driven translation from BPEL + contracts into a Spring Boot + Maven project using Camel/CXF, with deterministic mappings and centralised fault handling, verified via contract checks and real integration tests.

4.2 Feasibility and Importance

Feasibility is high because the assets are contract-first and the BPEL encodes mediation steps. Importance lies in replacing legacy BPEL with modern Java micro-integration while preserving behaviour, improving maintainability, and enabling reproducible automation.

4.3 Challenges and Fine Points

IBM-specific BPEL features (BOFactory, SDO) must be mapped to plain JAXB + Spring. Some imports may be missing, and business rule nodes (e.g., fire district checks) require explicit specifications. End-to-end tests depend on live partner endpoints; when unavailable, tests must be reported as blocked, never simulated.

4.4 Used Technology

- **Local agents orchestrated with Ollama and LangGraph** — Lightweight, on-premise LLM orchestration and graph-based agent coordination.
- **Java 17** — Long-term-support JVM release providing modern language features and performance.
- **Maven** — Declarative Java build and dependency-management tool.
- **Spring Boot** — Opinionated micro-service framework for rapid, production-ready Spring applications.
- **Apache Camel** — Enterprise-integration patterns engine for declarative routing and mediation.
- **Apache CXF** — Open-source services framework supporting JAX-WS/JAX-RS and SOAP/REST bindings.
- **JAXB/JAX-WS tooling** — Standard XML binding and web-service stub generation from WSDL/XSD.

- **MapStruct** — Compile-time code generator for type-safe, high-performance object mappings.
- **SLF4J/Logback** — Simple logging façade with fast, configurable log implementation.
- **JUnit** — Ubiquitous testing framework for unit and integration verification.
- **Shell/Python automation drivers** — Portable scripts executed inside the project virtual environment to drive agent workflows.
- **jq/yq** — CLI tools for JSON/YAML processing used by contract discovery and preflight reporting.
- **Docker Docker Compose** — Containerisation of services, clients, orchestration, FastAPI, and optional DB; local multi-service orchestration.
- **CICD (GitHub Actions or Jenkins)** — Deterministic pipeline executing preflight, build, test, packaging, and container publish.
- **Node.js React** — Frontend dashboard for evidence browsing and orchestration controls.
- **Python FastAPI** — Lightweight API for evidence metadata and agent orchestration hooks.

4.5 Local Models via Ollama

- Instruction/Planner model: use a general instruction-tuned model to plan tasks, enforce governance (no mocks, British English), and produce concise, action-oriented steps for agents.
- Code generation model: use a coder-oriented model to generate Java/Spring Boot, Maven, Apache Camel, and Apache CXF code with high syntactic accuracy and consistent patterns.
- Embedding/Retrieval model: use a high-quality text embedding model to index and retrieve WSDL/XSD contracts and code snippets for precise mapping and route wiring.

Table 2 details the recommended local models and their rationale.

Table 2: Recommended local models and rationale.

Model Role	**Recommended Model(s)**	**Rationale**
Instruction / Planner	Llama 3 Instruct (local)	Strong instruction-following aids planning, decomposition, and policy adherence (no mocks, British English), producing reliable agent steps and acceptance criteria.
Code Generation	CodeLlama or Qwen Coder (local)	Trained for programming tasks; produces idiomatic Java, Spring Boot, Maven, Camel, and CXF code with fewer syntax errors and better library usage patterns.

Embedding / Retrieval	nomic-embed-text or mxbai-embed-large (local)	High-quality embeddings for contract/code search, enabling deterministic mapping between WSDL/XSD schemas and generated classes, improving route wiring accuracy.

4.6 Agentic System Architecture

Agents: Planner, Contract Reader, BPEL Parser, Schema Aligner, Integration Builder (CXF/Camel), Mapping Builder (MapStruct), Test Designer, Environment Installer, Evaluator/Referee. Each agent has a focused remit and produces artefacts consumed downstream.

The Environment Installer performs a tooling preflight, verifies JDK/Maven and auxiliary tools (jq/yq, Python packages), installs missing dependencies where permitted, and emits a versioned readiness report at `doc/tooling-report.md`. It ensures all downstream agents can compile, run, capture logs, and evaluate correctness.

Figure 2 presents the agentic system pipeline, congruent with Table 3. Figure 3 depicts the high-level mediation flow derived from BPEL.

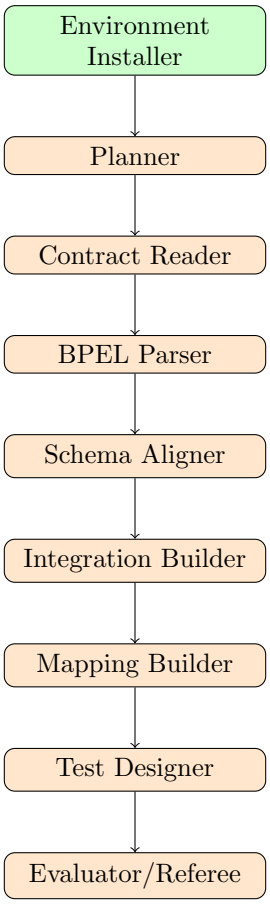

Figure 2: Agentic system pipeline congruent with the summary table.

4.7 Expected Results

A runnable Spring Boot application exposing LOC3X1B operations; generated CXF clients for partners; mappings that mirror BPEL assignments; centralised fault handling producing

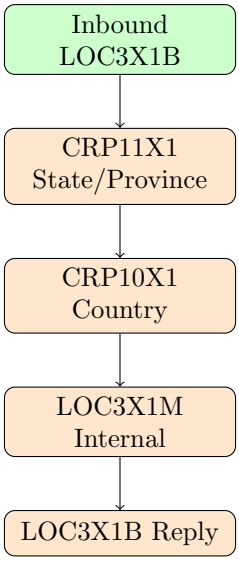

Figure 3: High-level flow derived from BPEL mediation.

SimpleFault; contract validations passing; integration tests either passing against live endpoints or clearly reported as blocked.

Note: The orchestration specification includes a `data_mappings` *section derived deterministically by parsing original IBM* `.map` *XML artefacts (e.g.,* `map:businessObjectMap` *and* `map:propertyMap`*). This reflects IBM BPEL script-based transformations rather than* `bpws:assign` *elements.*

5 Project Organisation

Recommended directory layout (generalised monorepo):

```
project-root/
|-- doc/                # LaTeX, tickets, plans, reports
|-- scripts/            # Automation (preflight, build, run, tests, test_master)
|-- contracts/          # Inventories, canonical maps, generated artefacts
|-- modules/            # Maven multi-module: contracts, services/loc3x1b, clients/{loc3x1m
|-- docker/             # Dockerfiles and docker-compose.yml
|-- api/                # FastAPI service (evidence, orchestration endpoints)
`-- tests/              # Test artefacts and JUnit XML outputs (mirrors sections)
```

6 Installation Scripts (Definition)

- `scripts/preflight.sh` – Runs tooling preflight, verifies JDK/Maven and CLI tools, produces `doc/tooling-report.md`, and attempts a smoke build if a Maven project is present.
- `scripts/bootstrap_java.sh` – Installs JDK and Maven locally so the build toolchain is ready.
- `scripts/generate_contracts.sh` – Runs JAXB/CXF codegen to create Java models and service stubs from WSDL/XSD.
- `scripts/build_all.sh` – Executes Maven compile, test and package for the entire project.

Table 3: Agent responsibilities, inputs, and outputs.

Agent	Responsibility	Inputs	Outputs
Planner	Global plan, dependency ordering, acceptance criteria alignment	BPEL, WSDL/XSD inventory	Execution plan, module breakdown
Contract Reader	Generate JAXB/CXF artefacts and validate contracts	WSDL/XSD	Models, stubs, contract validation report
BPEL Parser	Extract inbound ops, partner invocations, variables, fault logic	BPEL	Orchestration spec, mapping intents
Schema Aligner	Namespace/version governance and schema catalogue curation	XSD set	Canonical schema map, alignment report
Integration Builder	Configure CXF endpoints and Camel routes	Orchestration spec, properties	Running inbound service, route definitions
Mapping Builder	Deterministic request/response transformations	Orchestration spec, models	Mapper implementations and tests
Environment Installer	Provision local JDK/Maven and project tooling	Script definitions	Installed toolchain, verified versions
Evaluator/Referee	Verify builds, runtime, tests; produce final audit	Build outputs, test reports	Evaluation summary, blockers list

- `scripts/run_services.sh` – Starts the Spring-Boot inbound application exposing the LOC3X1B service.
- `scripts/test_integrations.sh` – Runs integration tests against live partner endpoints (no mocks).
- `scripts/test_master.sh` – Master test runner to execute unit tests by ordinal (LOC-XXX), by section, or all.

All scripts run inside the project virtual environment for consistent orchestration.

7 Agentic Strategy and Implementation Philosophy

7.1 Overview of the Agentic Approach

The transformation from BPEL-based integration to modern Spring Boot + Camel architecture employs an *agentic strategy* that fundamentally differs from traditional code generation or manual migration approaches. Rather than producing static artefacts that become obsolete when contracts change, this system implements a **closed-loop, evidence-driven integration layer** that continuously adapts to evolving service landscapes whilst maintaining operational stability and auditability.

The core philosophy centres on **dynamic resilience over static perfection**. The agentic system prioritises maintaining functional integrations even when partner services change locations, bindings, or schemas, rather than requiring manual intervention for every contract modification. This approach recognises that enterprise integration environments are inherently volatile, with services frequently updated, relocated, or temporarily unavailable.

7.2 The Closed-Loop Integration Cycle

The agentic system operates through a continuous seven-phase cycle: **Discover → Validate → Synthesise → Verify → Harden → Monitor → Adapt**. This cycle ensures that integration behaviour remains consistent whilst the underlying implementation adapts to environmental changes.

Figure 4 illustrates this closed-loop process, showing how each phase feeds into the next whilst maintaining evidence trails for governance and auditability.

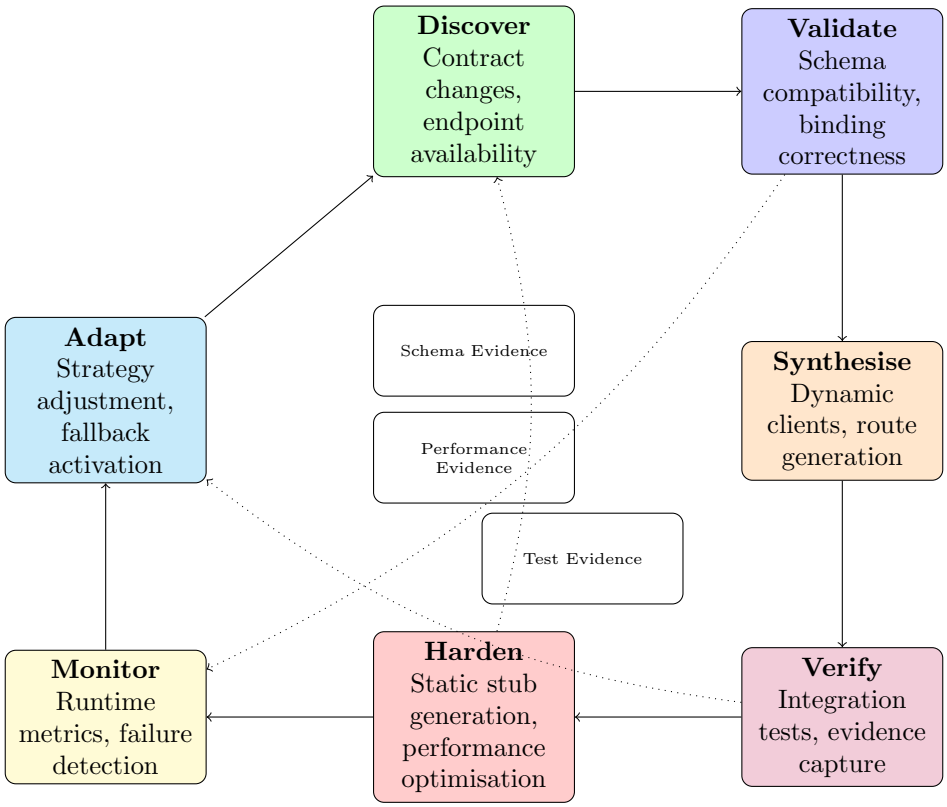

Figure 4: The closed-loop agentic integration cycle with evidence flows.

7.2.1 Phase Descriptions

- **Discover**: Continuously scans for changes in WSDL locations, schema versions, endpoint availability, and BPEL process definitions. Uses configurable polling intervals and change detection algorithms.
- **Validate**: Performs schema compatibility checks, binding validation, and import resolution. Identifies breaking changes versus compatible extensions.
- **Synthesise**: Generates integration artefacts using either static code generation (for stable contracts) or dynamic invocation patterns (for volatile contracts). Chooses strategy based on contract stability metrics.
- **Verify**: Executes targeted integration tests against live endpoints, capturing evidence of success or failure. Never uses mocks—blocked tests are reported as such.
- **Harden**: Converts proven dynamic integrations into optimised static implementations when stability thresholds are met. Maintains both versions during transition periods.

- **Monitor**: Tracks runtime performance, error rates, and availability metrics. Detects degradation patterns that trigger cycle re-entry.
- **Adapt**: Adjusts integration strategies based on accumulated evidence. May switch from static to dynamic approaches or activate fallback mechanisms.

7.3 Static versus Dynamic Invocation Strategy

The agentic system employs a **hybrid invocation model** that dynamically selects between static code generation and runtime service discovery based on contract stability and operational requirements.

Figure 5 depicts the decision matrix used to determine the optimal invocation approach for each service integration.

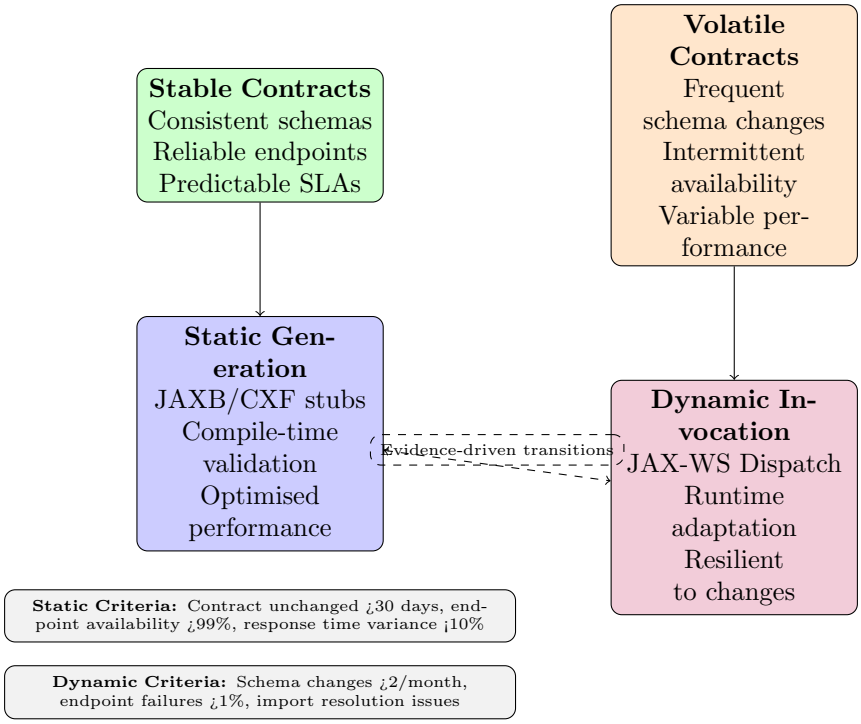

Figure 5: Static versus dynamic invocation decision matrix.

7.3.1 Static Generation Benefits

- **Performance**: Compile-time optimisation eliminates runtime reflection overhead
- **Type Safety**: Strong typing catches integration errors at build time
- **IDE Support**: Full code completion and refactoring capabilities
- **Debugging**: Stack traces reference actual Java classes rather than dynamic proxies

7.3.2 Dynamic Invocation Benefits

- **Adaptability**: Continues functioning when contracts change without recompilation
- **Resilience**: Graceful degradation when endpoints become temporarily unavailable
- **Discovery**: Automatic detection of new operations or modified schemas
- **Fallback**: Can switch between multiple endpoint versions transparently

7.4 Evidence-Driven Hardening Process

The transition from dynamic to static implementations follows a rigorous **evidence-driven hardening process** that ensures stability before committing to optimised static code generation.

7.4.1 Evidence Collection Metrics

- **Contract Stability**: Time since last schema change, frequency of endpoint modifications
- **Operational Reliability**: Success rate, response time consistency, availability percentage
- **Integration Complexity**: Number of transformations, fault handling requirements, orchestration depth
- **Business Criticality**: SLA requirements, transaction volume, downstream dependencies

7.4.2 Hardening Thresholds

Static generation is triggered when all of the following criteria are met for a continuous period:

- Contract unchanged for minimum 30 days
- Endpoint availability exceeding 99.5%
- Response time variance below 10% of baseline
- Zero import resolution failures
- Successful integration test rate above 98%

7.5 Orchestration Synthesis from BPEL

The agentic system transforms BPEL orchestration logic into Camel routes through a sophisticated **orchestration synthesis process** that preserves business logic whilst modernising the execution environment.

Figure 6 illustrates how BPEL constructs are mapped to equivalent Camel patterns whilst maintaining the original orchestration semantics.

7.5.1 Synthesis Algorithms

- **Partner Link Analysis**: Extracts service dependencies and generates corresponding CXF client configurations with appropriate timeout and retry policies
- **Variable Flow Tracking**: Maps BPEL variable assignments to type-safe MapStruct transformations, preserving data lineage and validation rules
- **Fault Propagation**: Converts BPEL fault handlers into Camel exception handling with appropriate compensation and rollback logic
- **Correlation Preservation**: Maintains message correlation patterns from BPEL correlation sets using Camel's correlation identifier mechanisms

7.6 Governance and Safety Mechanisms

The agentic system incorporates comprehensive **governance and safety mechanisms** to ensure that automated decisions remain auditable, reversible, and aligned with operational policies.

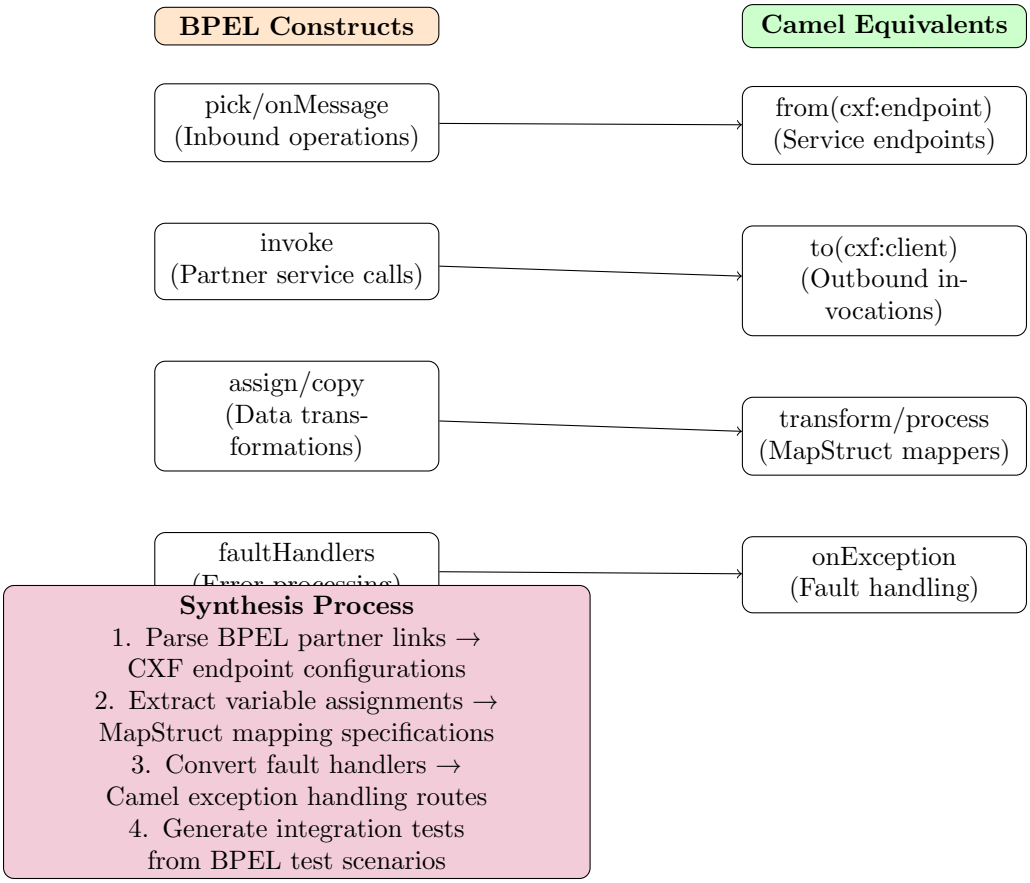

Figure 6: BPEL to Camel orchestration synthesis mapping.

7.6.1 Evidence Gates

Every significant system change must pass through **evidence gates** that require documented justification and approval criteria:

- **Contract Changes**: Schema compatibility analysis, impact assessment, rollback procedures
- **Endpoint Modifications**: Availability verification, performance benchmarking, fallback validation
- **Route Alterations**: Integration test results, business logic preservation, monitoring coverage
- **Hardening Decisions**: Stability evidence, performance improvements, maintenance implications

7.6.2 Auditability Requirements

All agentic decisions are logged with sufficient detail for post-hoc analysis:

- **Decision Context**: Environmental conditions, available evidence, applied policies
- **Alternative Considerations**: Other strategies evaluated, rejection rationale
- **Implementation Details**: Generated artefacts, configuration changes, test results
- **Success Metrics**: Performance improvements, reliability gains, maintenance reductions

7.6.3 Safety Constraints

The system operates within strict safety boundaries to prevent operational disruption:

- **No Secret Exposure**: Credentials and sensitive configuration never logged or committed
- **Gradual Rollout**: Changes deployed incrementally with monitoring at each stage
- **Automatic Rollback**: Failed changes trigger immediate reversion to last known good state
- **Human Override**: Manual intervention capabilities for emergency situations

7.7 Production Operational Model

In production environments, the agentic system operates as a **self-healing integration layer** that maintains service continuity whilst adapting to changing conditions.

7.7.1 Continuous Monitoring

- **Contract Drift Detection**: Automated scanning for WSDL/XSD changes with impact analysis
- **Performance Baseline Tracking**: Statistical analysis of response times, throughput, and error rates
- **Availability Monitoring**: Endpoint health checks with intelligent retry and circuit breaker patterns
- **Integration Quality Metrics**: Success rates, data quality scores, business rule compliance

7.7.2 Adaptive Response Strategies

When environmental changes are detected, the system employs graduated response strategies:

1. **Transparent Adaptation**: Minor changes handled automatically without service interruption
2. **Graceful Degradation**: Temporary fallbacks activated whilst permanent solutions are synthesised
3. **Controlled Failover**: Systematic switching to alternative endpoints or service versions
4. **Emergency Isolation**: Problematic integrations quarantined to prevent cascade failures

7.7.3 Expected Production Outcomes

The agentic approach delivers several key operational benefits:

- **Reduced Maintenance Overhead**: Automatic adaptation eliminates manual intervention for routine changes
- **Improved Reliability**: Multiple fallback strategies and proactive monitoring prevent service disruptions
- **Enhanced Visibility**: Comprehensive evidence trails enable rapid problem diagnosis and resolution

- **Accelerated Integration**: New services can be integrated rapidly using proven patterns and automated testing

This agentic strategy represents a fundamental shift from reactive maintenance to proactive adaptation, enabling integration architectures that evolve gracefully with their operational environments whilst maintaining the reliability and auditability required for enterprise systems.

8 Conclusion

The transformation is feasible and valuable. With disciplined agentic automation and strict adherence to the original contracts and BPEL intent, the resulting project will be maintainable, verifiable, and aligned with modern Java integration practices.